

DOCUMENT ARCHITECTURE



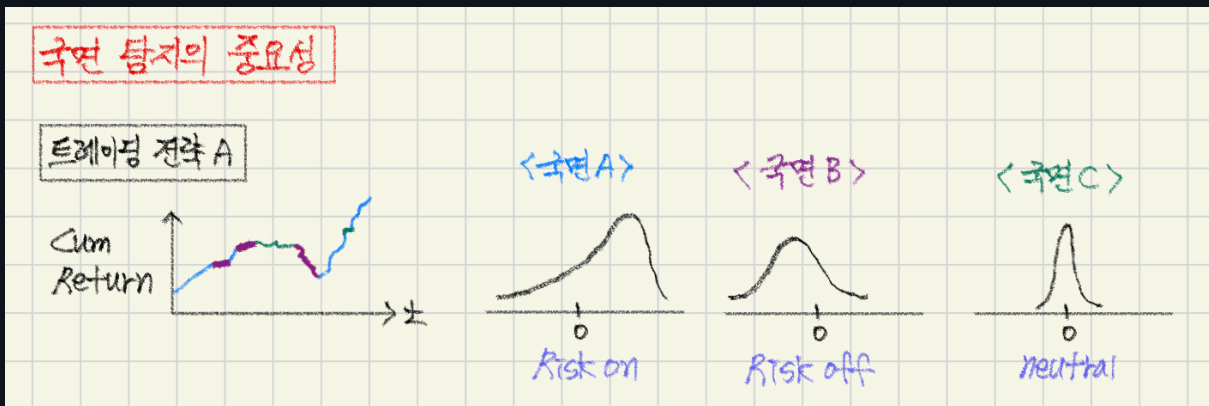
Hidden Markov Model 관련 이론 정리

Quant : Importance of Regime Detection

- 특정 전략을 활용한 트레이딩 과정에서 직면하는 어려움 중 하나는

정부의 정책 변화, 규제 환경 변화, 기타 거시 경제적 요인 변화로 인한 국면 변화로 인해 금융 시장의 행동이 자주, 갑작스럽게 바뀐다는 점이다.

- 국면에 따라서 자산 수익률의 분포가 달라지기 때문에 국면을 효과적으로 탐지하고 분류하여, 트레이딩 전략을 언제 어떻게 사용할지 결정하고, 그 전략의 파라미터를 최적으로 조정할 필요성이 생긴다.
- 즉, 이 점은 전략의 전체 생애주기 동안 전략을 어떻게 수정해야 하는지에 매우 중요한 영향을 준다.



- 예시
 - 전반적으로 트레이딩 전략 A는 우상향하는 모습을 보여 좋은 전략처럼 느껴진다. 하지만 국면 별로 수익률 분포가 다르기 때문에 국면의 특징에 따라서 베팅 사이즈를 조절할 필요성이 생긴다.
 - 국면 A : 평균 수익률이 크고 변동성도 크다. 이는 기대수익이 위험을 보상해주는 국면이므로 Risk-on mode 라고 할 수 있다. 따라서 베팅 사이즈를 증가 시켜야 한다.
 - (모멘텀, 베타 전략 선호)
 - 국면 B : 전반적으로 손실의 비중이 높으므로 Risk off mode 라고 할 수 있으며, 베팅 사이즈를 감소 시켜야 한다.
 - (현금, 채권, 헷지 선호)
 - 국면 C : 대부분 수익률이 0 근처에 위치하므로 방향성이 없는 장세이다.

- (캐리, 평균 회귀 전략 선호)

Hidden Markov Model : HMM

하이퍼 파라미터 기호 정리

- State의 개수 : k
- Sequence : $1 \sim T$
- Observation : $1 \sim N$

Markov Model 과 State Transition Probability Matrix

마르코프 모델 정의

- State의 Sequence를 상태 전이 확률 행렬로 표현하는 것.

상태 전이 확률 행렬

상태 전이 확률 행렬

Ex. State = $\{S_1, S_2\}$

1) State sequence : $S_1 \rightarrow S_2 \rightarrow S_2 \rightarrow S_2 \rightarrow S_1 \rightarrow S_1 \rightarrow S_2$

2) State transition probability Matrix (A) $\rightarrow$ If state num = k , $A \in \mathbb{R}^{k \times k}$

		To	
		S_1	S_2
from	S_1	1	2
	S_2	2	2

$\xrightarrow{\div \text{row_sum}}$

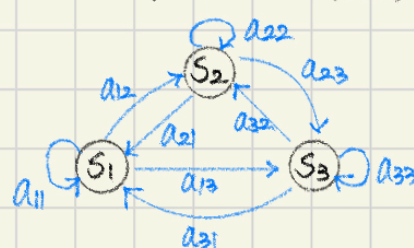
		To	
		S_1	S_2
from	S_1	$1/3$	$2/3$
	S_2	$1/3$	$2/3$

3) 상태 전이 확률 행렬 2가지 시각화 방법

Matrix form : $A(a_{ij})$

		to		
		S_1	S_2	S_3
from	S_1	a_{11}	a_{12}	a_{13}
	S_2	a_{21}	a_{22}	a_{23}
	S_3	a_{31}	a_{32}	a_{33}

Graph form : $A(a_{ij})$



✦ Markov Assumption(마르코프 가정)

정의

- 시간 t 에서의 관측값은 가장 최근 r 개의 관측에만 의존 한다는 가정
- 즉, 한 상태에서 다른 상태로의 전이는 이전 상태의 긴 이력을 필요하지 않다는 가정

Markov 가정 시각화

EX. $P=1$: $P(s_t | s_{t-1}, \dots, s_1) = P(s_t | s_{t-1})$ $0 \rightarrow 0 \rightarrow 0 \rightarrow 0$

EX. $P=2$: $P(s_t | s_{t-1}, s_{t-2}, \dots, s_1) = P(s_t | s_{t-1}, s_{t-2})$ $0 \rightarrow 0 \rightarrow 0 \rightarrow 0$

✦ HMM의 기본 가정

HMM 모형에서 필요한 2가지 Sequence Data

1. Hidden State Sequence(S) : HMM으로 추론하고자 하는 대상

$$S = (s_1, s_2, \dots, s_T)$$

2. Observation Sequence(Y) : 실제 우리가 관측할 수 있는 데이터

$$Y = (y_1, y_2, \dots, y_T)$$

HMM 모형의 가정

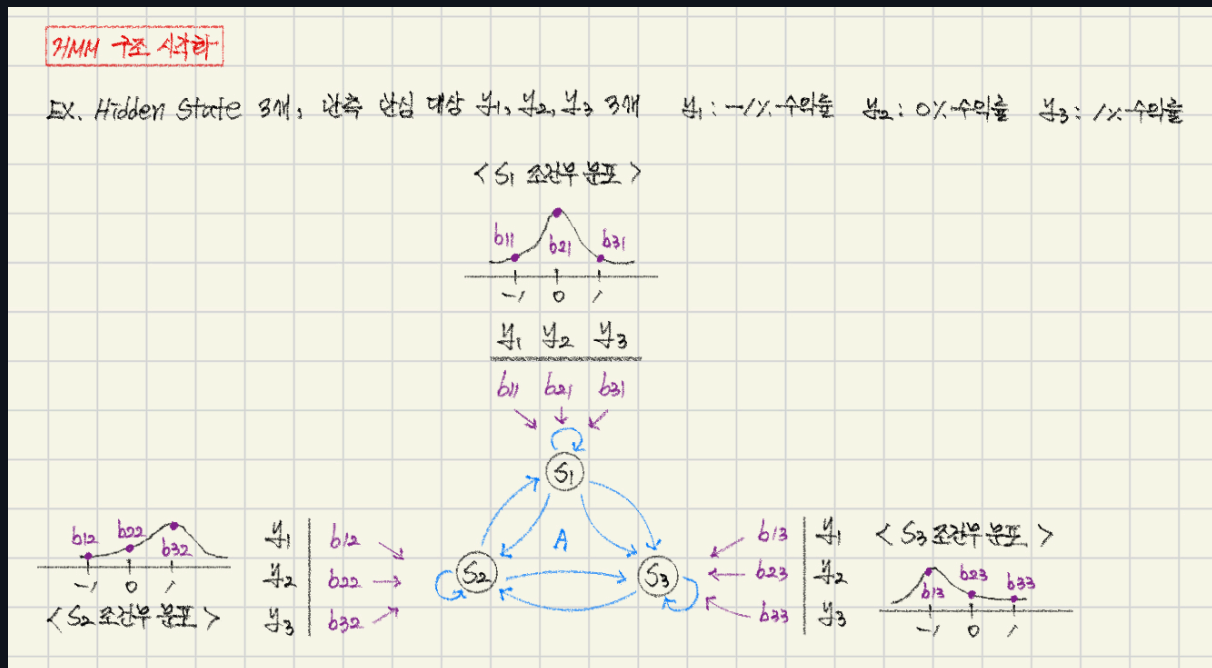
- 우리 눈에 보이지 않는 상태 시퀀스(hidden state sequence)가 보이는 관측 시퀀스(observation sequence)를 생성한다.

- 이때 hidden state sequence는 Markov Assumption을 따르며 observation sequence는 state conditional distribution을 따른다.

$$y_t \mid s_t \sim P(\cdot \mid s_t) \quad s_t \mid s_{t-1}, \dots, s_{t-r} \sim \text{Markov}$$

- 따라서 보이는 관측 시퀀스를 활용 하여 눈에 보이지 않는 Hidden State Sequence를 확률적으로 유추 하는 것이 HMM 모형의 목적이다.

HMM 모형의 구조



HMM 모형의 파라미터 ($\Lambda = A, B, \Pi$)

1. State Transition Probability Matrix (상태전이확률 행렬)

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \quad a_{ij} : i\text{번째 hidden state에서 } j\text{번째 hidden state로 전이}$$

$$a_{ij} = P(S_t = s_j \mid S_{t-1} = s_i) \quad 1 \leq i, j \leq k$$

$$\sum_{j=1}^k a_{ij} = 1 \rightarrow \text{row-sum} = 1$$

2. Emission Probability Matrix (방출확률 행렬)

$$B = \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{31} & b_{32} & b_{33} \end{bmatrix} \quad b_{ij} : j\text{번째 hidden state 에서 보이는 관측값 } i \text{가 생성될}$$

$$b_{ij} = P(Y_t = y_i \mid S_t = s_j) \quad 1 \leq j \leq k, \quad 1 \leq i \leq m$$

$$\sum_{i=1}^m b_{ij} = 1 \text{ or } \int_1^m P(y \mid S_t = s_j) dy = 1 \rightarrow \text{column-sum} = 1$$

1. Initial State Probability Matrix (초기상태확률 행렬)

$$\Pi = \begin{bmatrix} \pi_1 \\ \pi_2 \\ \pi_3 \end{bmatrix} \quad \pi_i : \text{sequence } t=0 \text{에서 } i\text{번째 hidden state 일 확률}$$

$$\sum_{i=1}^k \pi_i = 1$$

📌 Main Problems of HMM(Λ)

1. Learning Problem

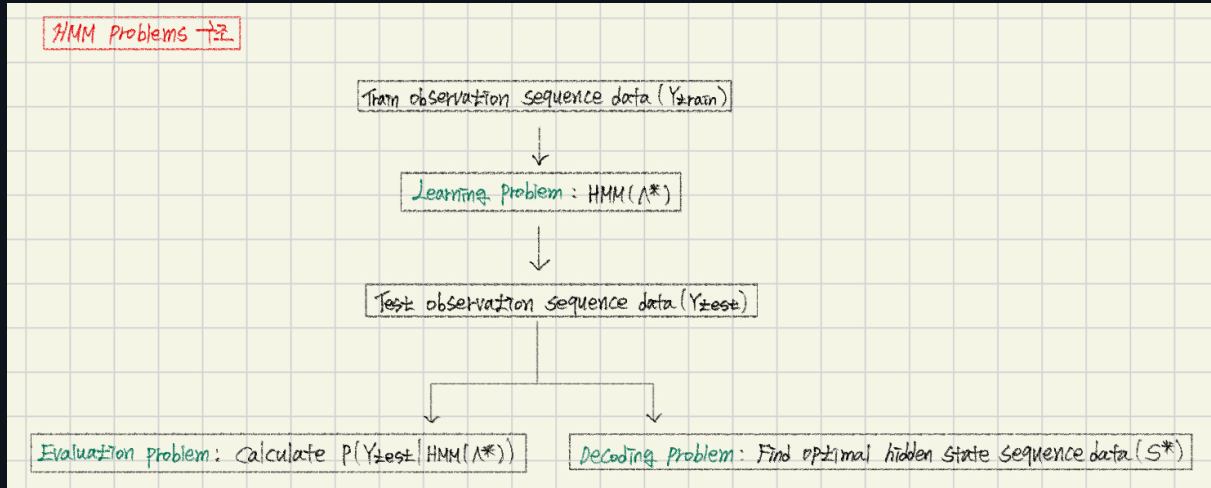
- Train Observation Sequence $Y = (y_1, y_2, \dots, y_T)$ 가 주어졌을 때, 최적의 HMM 파라미터 $\Lambda^* = \{A^*, B^*, \Pi^*\}$ 를 찾는 문제

2. Evaluation Problem

- 적합된 HMM(Λ^*)과 Test Observation Sequence Y 가 주어졌을 때, Test Observation Sequence Y 가 발생할 확률을 구하는 문제

3. Decoding Problem

- 적합된 HMM(Λ^*)과 Test Observation Sequence Y 가 주어졌을 때, Test Observation Sequence Y 에 대한 최적의 Hidden State Sequence를 찾는 문제



★ Learning Problem

정의

- Train Observation Sequence $Y = (y_1, y_2, \dots, y_T)$ 가 주어졌을 때, 최적의 HMM 파라미터 $\Lambda^* = \{A^*, B^*, \Pi^*\}$ 를 찾는 문제

Solution : Baum-Welch Algorithm (Forward - Backward Algorithm, Type of EM Algorithm)

1. Initialize HMM(Λ) architecture

2. Iterate until convergence or until the maximum number of iterations is reached

i. E-Step

- Calculate Forward / Backward Probability
- Calculate $\gamma_t(i)$ and $\xi_t(i, j)$

- Update : $\Pi^{new}, A^{new}, B^{new}$

iii.

$$| \log(1_k^T \cdot \alpha_T^{new}) - \log(1_k^T \cdot \alpha_T^{old}) | < \epsilon : \text{stop}$$

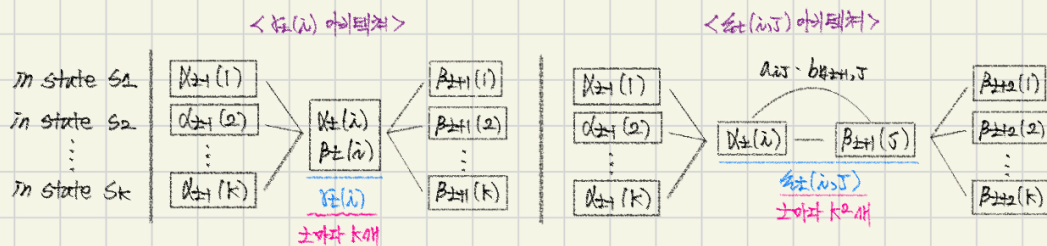
Solution of Learning Problem : Baum-Welch Algorithm

1 E-step: Forward / Backward algorithm으로 $\alpha_t(i), \beta_t(i)$ 구하기

2 E-step: $\xi_t(i), \xi_{t+1}(i, j)$ 구하기

$\xi_t(i)$: HMM(λ)과 Y_{train} 이 주어졌을 때, t 시점 state가 S_i 일 확률

$\xi_{t+1}(i, j)$: HMM(λ)과 Y_{train} 이 주어졌을 때, t 시점 state가 S_i , $t+1$ 시점 state가 S_j 일 확률



$$\xi_t(i) = P(S_t = S_i | \text{HMM}(\lambda), Y_{train}) = \frac{\alpha_t(i) \cdot \beta_t(i)}{\sum_{j=1}^k \alpha_t(j) \beta_t(j)} = \frac{\text{t에서 } S_i \text{ 일 확률}}{\text{t에서 } S_{1:k} \text{ 일 확률 모두 더한 값}}, 1 \leq t \leq T, 1 \leq i \leq k$$

$$\xi_{t+1}(i, j) = P(S_t = S_i, S_{t+1} = S_j | \text{HMM}(\lambda), Y_{train}) = \frac{\alpha_t(i) a_{ij} b_{j+1, j} \beta_{t+1}(j)}{\sum_{i=1}^k \sum_{j=1}^k \alpha_t(i) a_{ij} b_{j+1, j} \beta_{t+1}(j)}, 1 \leq t \leq T-1, 1 \leq i, j \leq k$$

= $\frac{\text{t에서 } S_i, \text{ t+1에서 } S_j \text{ 일 확률}}{\text{t에서 } S_{1:k} \text{ \& t+1에서 } S_{1:k} \text{의 조합 별 확률들 모두 더한 값}}$

3 M-step: Update $\Pi^{new}, A^{new}, B^{new}$

< Π^{new} update > $\rightarrow$ t에서 state가 S_i 일 확률이므로

$$\Pi_{ii}^{new} = \xi_1(i), 1 \leq i \leq k$$

< A^{new} update >

$$a_{ij}^{new} = \frac{\sum_{t=1}^T \xi_{t+1}(i, j)}{\sum_{t=1}^T \xi_t(i)}, 1 \leq i, j \leq k = \frac{\text{t=1~T까지 각각 시점에 } S_i \text{에서 } S_j \text{로 전이할 개수}}{\text{t=1~T까지 각각 시점에 } S_i \text{에서 전이할 개수}}$$

< B^{new} update >

$$b_{ij}^{new} = \frac{\sum_{t=1}^T \xi_t(i) \cdot Y_t = y_{ij} \cdot \xi_t(j)}{\sum_{t=1}^T \xi_t(i)}, 1 \leq j \leq k, 1 \leq i \leq N = \frac{\text{S}_i \text{에서 } y_{ij} \text{를 관측할 확률}}{\text{state가 } S_i \text{일 확률}}$$

정의

- 적합된 HMM(Λ^*)과 Test Observation Sequence Y 가 주어졌을 때, Test Observation Sequence Y 가 발생할 확률을 구하는 문제

Solution : Forward Algorithm

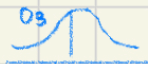
Forward Algorithm

EX. State = $\{S_1, S_2\}$ 2개 가정, 관심 observation = $\{O_1, O_2, O_3\}$ 3개 가정

1) HMM(Λ^*) & Test observation sequence (Y_{test})

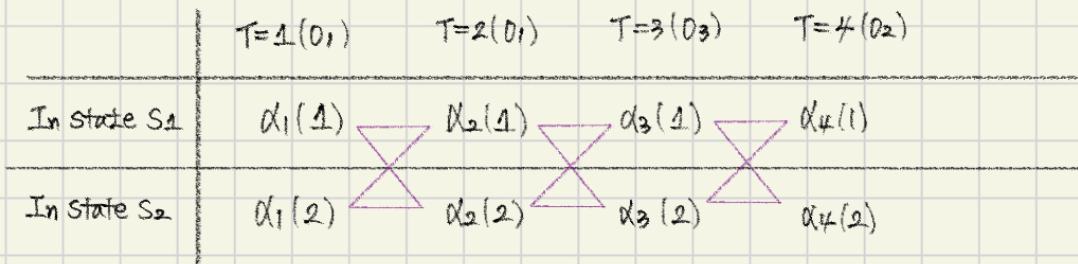
$$\pi = \begin{pmatrix} \pi_1 = 0.6 \\ \pi_2 = 0.4 \end{pmatrix} \quad A = \begin{pmatrix} a_{11} = 0.7 & a_{12} = 0.3 \\ a_{21} = 0.4 & a_{22} = 0.6 \end{pmatrix} \quad B = \begin{pmatrix} b_{11} = 0.1 & b_{12} = 0.6 \\ b_{21} = 0.4 & b_{22} = 0.3 \\ b_{31} = 0.5 & b_{32} = 0.1 \end{pmatrix}$$

$Y_{test} = (Y_1 = O_1, Y_2 = O_1, Y_3 = O_3, Y_4 = O_2)$

→ from  $\langle S_1 \text{ 조건부 분포} \rangle$

2) forward Algorithm

$\alpha_t(S_i)$ = 전방확률, 주어진 HMM(Λ^*), Y_{test} 하에서 t시점 State가 S_i 일 확률
이전 상태에서의 observation과 상태전이구조도 반영된 확률



1) $\alpha_1(1) = \pi_1 \cdot b_{11}$

$\alpha_1(2) = \pi_2 \cdot b_{12}$

2) $\alpha_2(1) = (\alpha_1(1) \cdot a_{11} + \alpha_1(2) \cdot a_{21}) \cdot b_{11}$

$\alpha_2(2) = (\alpha_1(1) \cdot a_{12} + \alpha_1(2) \cdot a_{22}) \cdot b_{12}$

상태 전이 구조 반영 Y_{test} 구조 반영

3) $\alpha_3(1) = (\alpha_2(1) \cdot a_{11} + \alpha_2(2) \cdot a_{21}) \cdot b_{31}$
 $\alpha_3(2) = (\alpha_2(1) \cdot a_{12} + \alpha_2(2) \cdot a_{22}) \cdot b_{32}$

4) $\alpha_4(1) = (\alpha_3(1) \cdot a_{11} + \alpha_3(2) \cdot a_{21}) \cdot b_{21}$
 $\alpha_4(2) = (\alpha_3(1) \cdot a_{12} + \alpha_3(2) \cdot a_{22}) \cdot b_{22}$

5) $P(Y_{test} | HMM(\Lambda^*)) = \alpha_4(1) + \alpha_4(2)$

Solution : Backward Algorithm

Backward Algorithm

Ex. State = $\{S_1, S_2\}$ 2개 가정, 관심 observation = $\{O_1, O_2, O_3\}$ 3개 가정

1. HMM(λ^*) & Test observation sequence (Y_{test})

$$\pi = \begin{pmatrix} \pi_1 = 0.6 \\ \pi_2 = 0.4 \end{pmatrix} \quad A = \begin{pmatrix} a_{11} = 0.7 & a_{12} = 0.3 \\ a_{21} = 0.4 & a_{22} = 0.6 \end{pmatrix} \quad B = \begin{pmatrix} b_{11} = 0.1 & b_{12} = 0.6 \\ b_{21} = 0.4 & b_{22} = 0.3 \\ b_{31} = 0.5 & b_{32} = 0.1 \end{pmatrix}$$

$$Y_{test} = (Y_1 = O_1, Y_2 = O_1, Y_3 = O_3, Y_4 = O_2)$$

→ from 
<S1 조건부 분포>

2. Backward Algorithm

$\beta_t(S_i)$ = 후방 확률, 주어진 HMM(λ^*), Y_{test} 하에서 t시점 state 가 S_i 일 확률
+ 이전 상태에서의 observation 과 상태전이 구조가 반영되지 않은 확률

	T=1(O_1)	T=2(O_1)	T=3(O_3)	T=4(O_2)
In state S_1	$\beta_1(1)$	$\beta_2(1)$	$\beta_3(1)$	$\beta_4(1)$
In state S_2	$\beta_2(2)$	$\beta_2(2)$	$\beta_3(2)$	$\beta_4(2)$

$$1) \beta_4(1) = \beta_4(2) = 1 \quad 2) \beta_3(1) = (\beta_4(1) \cdot a_{11} \cdot b_{21} + \beta_4(2) \cdot a_{12} \cdot b_{22})$$

$$\beta_3(2) = (\beta_4(1) \cdot a_{21} \cdot b_{21} + \beta_4(2) \cdot a_{22} \cdot b_{22})$$

다래 observation 과 상태전이 구조 반영

$$3) \beta_2(1) = (\beta_3(1) \cdot a_{11} \cdot b_{31} + \beta_3(2) \cdot a_{12} \cdot b_{32}) \quad 4) \beta_1(1) = (\beta_2(1) \cdot a_{11} \cdot b_{11} + \beta_2(2) \cdot a_{12} \cdot b_{12})$$

$$\beta_2(2) = (\beta_3(1) \cdot a_{21} \cdot b_{31} + \beta_3(2) \cdot a_{22} \cdot b_{32}) \quad \beta_1(2) = (\beta_2(1) \cdot a_{11} \cdot b_{11} + \beta_2(2) \cdot a_{12} \cdot b_{12})$$

$$5) P(Y_{test} | HMM(\lambda^*)) = (\beta_1(1) \cdot \pi_1 \cdot b_{11} + \beta_1(2) \cdot \pi_2 \cdot b_{12})$$

Forward / Backward Algorithm 수식 정리

• Forward Algorithm

$$\alpha_1(s_i) = \pi_i \cdot b_{y_1, i} \quad 1 \leq i \leq k$$

$$\alpha_t(s_i) = \left[\sum_{j=1}^k \alpha_{t-1}(s_j) \cdot a_{ji} \right] \cdot b_{y_t, i} \quad 2 \leq t \leq T \quad 1 \leq i \leq k$$

$$P(Y_{test} | HMM(\Lambda^*)) = \sum_{j=1}^k \alpha_T(j)$$

- Backward Algorithm

$$\beta_T(s_i) = 1 \quad 1 \leq i \leq k$$

$$\beta_t(s_i) = [\beta_{t+1}(s_j) \cdot a_{ij} \cdot b(y_{t+1}, j)] \quad 1 \leq i \leq T-1 \quad 1 \leq i \leq k$$

$$P(Y_{test} | HMM(\Lambda^*)) = \sum_{i=1}^k \beta_1(s_i) \cdot \pi_i \cdot b_{y_1, i}$$

Evaluation Problem 요약 정리

- Forward / Backward Algorithm으로 구한 각각의 $P(Y_{test} | HMM(\Lambda^*))$ 값은 같아야 한다.
- Evaluation Problem 은 Y_{test} 가 주어졌을 때 어떤 HMM 모형에서 나왔을 가능성이 더 높은지를 확인하는 Sequence Classification 문제에서 주로 활용된다.

📌 Decoding Problem

정의

- 적합한 $HMM(\Lambda^*)$ 과 Test Observation Sequence Y 가 주어졌을 때, Test Observation Sequence Y 에 대한 최적의 Hidden State Sequence를 찾는 문제

Solution : Viterbi Algorithm

- Viterbi Algorithm : $HMM(\Lambda^*)$ 에서 특정 Observation Sequence (Y_{test})가 주어졌을 때 가능성이 가장 높은 최적의 단일 Hidden State Sequence를 찾기 위한 다이나믹 프로그래밍(Dynamic Programming) 기법

Solution of Decoding problem: Viterbi Algorithm

Ex. state = $\{S_1, S_2\}$ 2개 가정, 관측 observation = $\{O_1, O_2, O_3\}$ 3개 가정

1) HMM(λ^*) & Test observation sequence (Y_{test})

$$\pi = \begin{pmatrix} \pi_1 = 0.6 \\ \pi_2 = 0.4 \end{pmatrix} \quad A = \begin{pmatrix} a_{11} = 0.7 & a_{12} = 0.3 \\ a_{21} = 0.4 & a_{22} = 0.6 \end{pmatrix} \quad B = \begin{pmatrix} b_{11} = 0.1 & b_{12} = 0.6 \\ b_{21} = 0.4 & b_{22} = 0.3 \\ b_{31} = 0.5 & b_{32} = 0.1 \end{pmatrix}$$

$$Y_{test} = (Y_1 = O_1, Y_2 = O_1, Y_3 = O_3, Y_4 = O_2)$$

from 
<S1 조인부 분포>

2) Viterbi Algorithm

$V_t(s_i)$ = t시점까지의 관측을 설명하는 가장 가능성 높은 상태 경로 중 t시점에 state가 S_i 로 끝나는 확률.

	T=1(O_1)	T=2(O_1)	T=3(O_3)	T=4(O_2)
In state S_1	$V_1(1)$	$V_2(1)$	$V_3(1)$	$V_4(1)$
In state S_2	$V_1(2)$	$V_2(2)$	$V_3(2)$	$V_4(2)$

- $V_1(1) = \pi_1 \cdot b_{11} = 0.06$
 $V_1(2) = \pi_2 \cdot b_{12} = 0.24$
 $\tau_1(1) = 0$ or None
 $\tau_1(2) = 0$ or None
 $\therefore \hat{Q} = (S_2, S_2, S_1, S_1)$
- $V_2(1) = \max(V_1(1) \cdot a_{11}, V_1(2) \cdot a_{21}) \cdot b_{11} = 0.0096$
 $V_2(2) = \max(V_1(1) \cdot a_{12}, V_1(2) \cdot a_{22}) \cdot b_{12} = 0.0864$
 $\tau_2(1) = 2$
 $\tau_2(2) = 2$
 $\hat{Q}_1 = \tau_2(2) = S_2$
- $V_3(1) = \max(V_2(1) \cdot a_{11}, V_2(2) \cdot a_{21}) \cdot b_{31} = 0.01728$
 $V_3(2) = \max(V_2(1) \cdot a_{12}, V_2(2) \cdot a_{22}) \cdot b_{32} = 0.00518$
 $\tau_3(1) = 2$
 $\tau_3(2) = 2$
 $\hat{Q}_2 = \tau_3(1) = S_2$
- $V_4(1) = \max(V_3(1) \cdot a_{11}, V_3(2) \cdot a_{21}) \cdot b_{21} = 0.00484$
 $V_4(2) = \max(V_3(1) \cdot a_{12}, V_3(2) \cdot a_{22}) \cdot b_{22} = 0.00156$
 $\tau_4(1) = 1$
 $\tau_4(2) = 1$
 $\hat{Q}_3 = \tau_4(1) = S_1$
- $\hat{Q}_{T=4} = \arg \max_{\hat{Q}} V_T(\hat{Q}) = S_1$

Viterbi Algorithm 수식 정리

$$v_1(s_i) = \pi_i \cdot b_{y_1, i} \quad \tau_1(s_i) = 0 \text{ or None} \quad 1 \leq i \leq k$$

$$v_t(s_i) = \max_{1 \leq j \leq k} v_{t-1}(s_j) \cdot a_{ji} \cdot b_{y_t, i} \quad \tau_t(s_i) = \arg \max_{1 \leq j \leq k} v_{t-1}(s_j) \cdot a_{ji} \quad 2 \leq t \leq T$$

$$\hat{q}_T = \arg \max_{1 \leq j \leq k} v_T(j)$$

$$\hat{q}_t = \tau_{t+1}(\hat{q}_{t+1}) \quad t = T-1, T-2, \cdots, 1$$

$$\hat{Q} = (\hat{q}_1, \hat{q}_2, \cdots, \hat{q}_T)$$

HMM : 시점 별 Hidden State의 확률 분포를 구하는 방법

★ Filtering

$$P(s_t | y_{1:t})$$

- 목적 : 시점 t 까지의 Observation Sequence ($y_{1:t}$)와 $HMM(\Lambda^*)$ 가 주어졌을 때, 현재 상태 s_t 의 확률 분포 $P(s_t | y_{1:t})$ 를 구하는 것
- 활용 : 주로 실시간 추적, 실시간 반응 시스템을 설계하기 위해 사용
- 알고리즘 : Forward Algorithm

Filtering

시점 t 의 state j 의 전방확률
시점 t 모든 상태의 전방확률 합

$$f_t(j) = \frac{\text{시점 } t \text{의 state } j \text{의 전방확률}}{\text{시점 } t \text{ 모든 상태의 전방확률 합}}$$
$$\therefore f_t(j) = \frac{\alpha_t(j)}{\sum_{i=1}^K \alpha_t(i)}$$

EX. state = $\{s_1, s_2\}$

	시점 t 전방확률
In state s_1	$\alpha_t(1)$
In state s_2	$\alpha_t(2)$

→

$$f_t(1) = \frac{\alpha_t(1)}{\alpha_t(1) + \alpha_t(2)}$$
$$f_t(2) = \frac{\alpha_t(2)}{\alpha_t(1) + \alpha_t(2)}$$

★ Smoothing

$$P(s_t | y_{1:T})$$

- 목적 : 전체 기간의 Observation Sequence ($y_{1:T}$)와 $HMM(\Lambda^*)$ 가 주어졌을 때, 특정 과거 시점의 상태 s_t 의 확률 분포 $P(s_t | y_{1:T})$ 를 구하는 것
- 활용 : 미래의 정보까지 활용하여 Filtering 보다 매끄럽고(Smooth) 정확한 추정을 하고자 할 때 사용
- 알고리즘 : Forward - Backward Algorithm

Smoothing

$$sm_{\pm}(j) = \frac{\text{특정 state } j \text{의 전방 확률} \times \text{후방 확률}}{\text{특정 모든 상태의 전방 확률} \times \text{후방 확률}}$$

$$\therefore sm_{\pm}(j) = \frac{a_{\pm}(j) \cdot b_{\pm}(j)}{\sum_{i=1}^K a_{\pm}(i) \cdot b_{\pm}(i)}$$

Ex. state = $\{s_1, s_2\}$

	특정 전방 확률
In state s_1	$a_{\pm}(1), b_{\pm}(1)$
In state s_2	$a_{\pm}(2), b_{\pm}(2)$

$$sm_{\pm}(1) = \frac{a_{\pm}(1) \cdot b_{\pm}(1)}{a_{\pm}(1) \cdot b_{\pm}(1) + a_{\pm}(2) \cdot b_{\pm}(2)}$$

$$sm_{\pm}(2) = \frac{a_{\pm}(2) \cdot b_{\pm}(2)}{a_{\pm}(1) \cdot b_{\pm}(1) + a_{\pm}(2) \cdot b_{\pm}(2)}$$

Prediction

$$P(s_{t+h} | y_{1:t})$$

- 목적 : 시점 t 까지의 Observation Sequence ($y_{1:t}$)와 $HMM(\Lambda^*)$ 가 주어졌을 때, 미래 시점 $t+h$ 의 상태 s_{t+h} 의 확률 분포 $P(s_{t+h} | y_{1:t})$ 를 구하는 것
- 활용 : 미래 매크로 환경 예측과 같이 향후 시장 상황을 전망하고자 할 때 사용
- 알고리즘 : Forward Algorithm + State Transition Probability Matrix (A)

Prediction

$$Pr_{t+h}(j) = (A^h)^T \cdot \vec{f}_t \text{의 } j\text{-번째 Component} \quad \text{where } \vec{f}_t = \begin{pmatrix} f_t(1) \\ \vdots \\ f_t(k) \end{pmatrix} \in \mathbb{R}^{k \times 1}$$

Ex. state = $\{s_1, s_2\}$

$$A = \begin{pmatrix} a_{11}=0.2 & a_{12}=0.8 \\ a_{21}=0.4 & a_{22}=0.6 \end{pmatrix}$$

one step prediction

$$Pr_{t+1} = A^T \cdot \vec{f}_t = \begin{pmatrix} a_{11} & a_{21} \\ a_{12} & a_{22} \end{pmatrix} \cdot \begin{pmatrix} f_t(1) \\ f_t(2) \end{pmatrix} = \begin{pmatrix} f_t(1) \cdot a_{11} + f_t(2) \cdot a_{21} \\ f_t(1) \cdot a_{12} + f_t(2) \cdot a_{22} \end{pmatrix}$$

$$\vec{f}_t = \begin{pmatrix} f_t(1) \\ f_t(2) \end{pmatrix}$$

→ 필터링 기댓값에 상태전이율 반영

🔍 Multivariate HMM : Observation Sequence의 종류가 여러가지인 경우

- **목표** : m ($m \geq 2$)개 Observation Sequence가 HMM Input으로 들어올 때 단변량 HMM과 어떤 차이가 있는지 파악하기
- **특징**
 - 상태 전이 확률 행렬(A), 초기 상태 확률 행렬(Π)은 변함이 없고,
 - 각각의 Observation Sequence 별로 방출 확률 행렬($B^{(m)}$)을 **factorization** 한 후,
 - **행렬곱** 과 **element wise product** 로 간단하게 표현할 수 있음.

📌 Multivariate Forward Algorithm

알고리즘 정리

- ○ : Element Wise Product

$$(t = 1) \quad \alpha_1 = \Pi \circ \bigcirc_{i=1}^m (B_{\text{row}(Y_1^{(i)})}^{(i)})^T$$

$$(t = 2 \sim T) \quad \alpha_t = (A^T \cdot \alpha_{t-1}) \circ \bigcirc_{i=1}^m (B_{\text{row}(Y_t^{(i)})}^{(i)})^T$$

$$(t = T) \quad \text{Likelihood} = P(Y_{\text{test}} \mid HMM(\Lambda^*)) = 1^T \cdot \alpha_T$$

예시

Multivariate forward Algorithm

EX. state = $\{s_1, s_2\}$, m 개의 이산 연속 시퀀스 존재 $Y_{1:3} = \begin{pmatrix} Y_1 & Y_2 & Y_3 \\ y_1^{(1)} & y_2^{(1)} & y_3^{(1)} \\ y_1^{(2)} & y_2^{(2)} & y_3^{(2)} \end{pmatrix}$

$$A = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \quad \pi = \begin{pmatrix} \pi_1 \\ \pi_2 \end{pmatrix} \quad B^{(1)} = \begin{pmatrix} b_{11}^{(1)} & b_{12}^{(1)} \\ b_{21}^{(1)} & b_{22}^{(1)} \\ b_{31}^{(1)} & b_{32}^{(1)} \end{pmatrix} \quad B^{(2)} = \begin{pmatrix} b_{11}^{(2)} & b_{12}^{(2)} \\ b_{21}^{(2)} & b_{22}^{(2)} \\ b_{31}^{(2)} & b_{32}^{(2)} \end{pmatrix}$$

$$\begin{aligned} (\pm=1) \quad \alpha_1(1) &= \pi_1 \times \underbrace{(b_{11}^{(1)} \times b_{21}^{(2)})}_{\text{독립성 가정}} \\ \alpha_1(2) &= \pi_2 \times \underbrace{(b_{12}^{(1)} \times b_{22}^{(2)})}_{\text{독립성 가정}} \end{aligned} \Rightarrow \alpha_1 = \begin{pmatrix} \alpha_1(1) \\ \alpha_1(2) \end{pmatrix} = \begin{pmatrix} \pi_1 \\ \pi_2 \end{pmatrix} \circ \begin{pmatrix} b_{11}^{(1)}, 1 \\ b_{11}^{(1)}, 2 \end{pmatrix} \circ \begin{pmatrix} b_{12}^{(2)}, 1 \\ b_{12}^{(2)}, 2 \end{pmatrix}$$

$$\therefore \alpha_1 = \pi \circ (B_{\text{row}(Y_1)}^{(1)})^T \circ (B_{\text{row}(Y_2)}^{(2)})^T \quad \therefore \alpha_1 = \pi \circ \prod_{i=1}^m (B_{\text{row}(Y_i)}^{(i)})^T$$

$$(\pm=2) \quad \alpha_2(1) = (\alpha_1(1) \cdot a_{11} + \alpha_1(2) \cdot a_{21}) \circ (b_{21}^{(1)} \cdot b_{31}^{(2)})$$

$$\alpha_2(2) = (\alpha_1(1) \cdot a_{12} + \alpha_1(2) \cdot a_{22}) \circ (b_{22}^{(1)} \cdot b_{32}^{(2)})$$

$$\alpha_2 = \begin{pmatrix} \alpha_2(1) \\ \alpha_2(2) \end{pmatrix} = \begin{pmatrix} a_{11} & a_{21} \\ a_{12} & a_{22} \end{pmatrix} \times \begin{pmatrix} \alpha_1(1) \\ \alpha_1(2) \end{pmatrix} \circ \begin{pmatrix} b_{21}^{(1)}, 1 \\ b_{21}^{(1)}, 2 \end{pmatrix} \circ \begin{pmatrix} b_{22}^{(2)}, 1 \\ b_{22}^{(2)}, 2 \end{pmatrix}$$

$$\therefore \alpha_2 = A^T \cdot \alpha_1 \circ (B_{\text{row}(Y_2)}^{(1)})^T \circ (B_{\text{row}(Y_3)}^{(2)})^T$$

$$\therefore \alpha_{\pm} = A^T \alpha_{\pm-1} \circ \prod_{i=1}^m (B_{\text{row}(Y_i)}^{(i)})^T$$

$$(\pm=T) \quad P(Y_{\text{test}} | \text{HMM}(\Lambda^*)) = \alpha_T(1) + \alpha_T(2) \quad \therefore \text{Likelihood: } P(Y_{\text{test}} | \text{HMM}(\Lambda^*)) = \mathbf{1}^T \alpha_T$$

★ Multivariate Backward Algorithm

알고리즘 정리

$$(t=T) \quad \beta_T = 1$$

$$(t=(T-1) \sim 1) \quad \beta_t = A \cdot (\beta_{t+1} \circ \bigcirc_{i=1}^m (B_{\text{row}(Y_{t+1}^{(i)})}^{(i)})^T)$$

$$(t = 1) \quad \text{Likelihood} = P(Y_{\text{test}} | HMM(\Lambda^*)) = \Pi^T \cdot (\beta_1 \circ \bigcirc_{i=1}^m (B_{\text{row}(Y_1^{(i)})}^{(i)})^T)$$

예시

Multivariate Backward Algorithm

Ex. state = $\{s_1, s_2\}$, $m (=2)$ 개의 관측 시퀀스 존재 $Y_{1:3} = \left(\begin{array}{c} Y_1 \\ Y_2 \\ Y_3 \end{array} \right) = \left(\begin{array}{c} y_1^{(1)} \\ y_2^{(1)} \\ y_3^{(1)} \end{array} \right) \left(\begin{array}{c} y_1^{(2)} \\ y_2^{(2)} \\ y_3^{(2)} \end{array} \right) \left(\begin{array}{c} y_1^{(3)} \\ y_2^{(3)} \\ y_3^{(3)} \end{array} \right)$

$$A = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \quad \Pi = \begin{pmatrix} \pi_1 \\ \pi_2 \end{pmatrix} \quad B^{(1)} = \begin{pmatrix} b_{11}^{(1)} & b_{12}^{(1)} \\ b_{21}^{(1)} & b_{22}^{(1)} \\ b_{31}^{(1)} & b_{32}^{(1)} \end{pmatrix} \quad B^{(2)} = \begin{pmatrix} b_{11}^{(2)} & b_{12}^{(2)} \\ b_{21}^{(2)} & b_{22}^{(2)} \\ b_{31}^{(2)} & b_{32}^{(2)} \end{pmatrix}$$

$$(\pm = T) \quad \beta_3 = \begin{pmatrix} \beta_3(1) \\ \beta_3(2) \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \end{pmatrix} \quad \therefore \beta_T = 1$$

$$(\pm = T-1)$$

$$\beta_2(1) = \beta_3(1) \cdot a_{11} \cdot b_{Y_3^{(1)},1}^{(1)} + \beta_3(2) \cdot a_{12} \cdot b_{Y_3^{(1)},2}^{(1)} \cdot b_{Y_3^{(2)},1}^{(1)}$$

$$\beta_2(2) = \beta_3(1) \cdot a_{21} \cdot b_{Y_3^{(1)},1}^{(1)} + \beta_3(2) \cdot a_{22} \cdot b_{Y_3^{(1)},2}^{(1)} \cdot b_{Y_3^{(2)},2}^{(1)}$$

$$\therefore \beta_2 = \begin{pmatrix} \beta_2(1) \\ \beta_2(2) \end{pmatrix} = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \cdot \begin{pmatrix} \beta_3(1) \\ \beta_3(2) \end{pmatrix} \circ \begin{pmatrix} b_{Y_3^{(1)},1}^{(1)} \\ b_{Y_3^{(1)},2}^{(1)} \end{pmatrix} \circ \begin{pmatrix} b_{Y_3^{(2)},1}^{(1)} \\ b_{Y_3^{(2)},2}^{(1)} \end{pmatrix}$$

$$\therefore \beta_2 = A \cdot (\beta_3 \circ B_{\text{row}(Y_3^{(1)})}^{(1)T} \circ B_{\text{row}(Y_3^{(2)})}^{(1)T})$$

$$\therefore \beta_{\pm} = A \cdot (\beta_{\pm+1} \circ \prod_{i=\pm+1}^m B_{\text{row}(Y_{\pm+1}^{(i)})}^{(i)T})$$

$$(\pm = 1)$$

$$P(Y_{\text{test}} | HMM(\Lambda^*)) = \pi_1 \cdot b_{Y_1^{(1)},1}^{(1)} \cdot b_{Y_1^{(2)},1}^{(1)} + \pi_2 \cdot b_{Y_1^{(1)},2}^{(1)} \cdot b_{Y_1^{(2)},2}^{(1)}$$

$$\therefore P(Y_{\text{test}} | HMM(\Lambda^*)) = (\pi_1 \quad \pi_2) \cdot \left(\begin{pmatrix} \pi_1(1) \\ \pi_1(2) \end{pmatrix} \circ \begin{pmatrix} b_{Y_1^{(1)},1}^{(1)} \\ b_{Y_1^{(2)},1}^{(1)} \end{pmatrix} \circ \begin{pmatrix} b_{Y_1^{(2)},1}^{(1)} \\ b_{Y_1^{(2)},2}^{(1)} \end{pmatrix} \right)$$

$$\therefore \text{Likelihood} : P(Y_{\text{test}} | HMM(\Lambda^*)) = \Pi^T \cdot (\beta_1 \circ B_{\text{row}(Y_1^{(1)})}^{(1)T} \circ B_{\text{row}(Y_1^{(2)})}^{(1)T})$$

$$\therefore \text{Likelihood} : P(Y_{\text{test}} | HMM(\Lambda^*)) = \Pi^T \cdot (\beta_1 \circ \prod_{i=1}^m B_{\text{row}(Y_i^{(i)})}^{(i)T})$$

★ Multivariate Viterbi Algorithm

$$(t = 1) \quad v_1 = \Pi \circ \bigcirc_{i=1}^m (B_{row(Y_1^{(i)})}^{(i)})^T$$

$$(t = 2 \sim T) \quad v_t = \text{rowmax}(A^T \cdot \text{Diag}(v_{t-1})) \circ \bigcirc_{i=1}^m (B_{row(Y_t^{(i)})}^{(i)})^T \quad \tau_t = \text{rowargmax}(v_t)$$

$$(\text{Backtracking at } T) \quad \hat{q}_T = \arg \max_i v_T(i)$$

$$(\text{Backtracking at } (T - 1) \sim 1) \quad \hat{q}_t = \tau_{t+1}(\hat{q}_{t+1})$$

예시

Multivariate Viterbi Algorithm

EX. state = $\{s_1, s_2\}$, $m (=2)$ 개의 관측 시퀀스 존재 $Y_{1:3} = \left(\begin{pmatrix} y_1^{(1)} \\ y_2^{(1)} \\ y_3^{(1)} \end{pmatrix} \begin{pmatrix} y_1^{(2)} \\ y_2^{(2)} \\ y_3^{(2)} \end{pmatrix} \begin{pmatrix} y_1^{(3)} \\ y_2^{(3)} \\ y_3^{(3)} \end{pmatrix} \right)$

$$A = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \quad \pi = \begin{pmatrix} \pi_1 \\ \pi_2 \end{pmatrix} \quad b^{(1)} = \begin{pmatrix} b_{11}^{(1)} & b_{12}^{(1)} \\ b_{21}^{(1)} & b_{22}^{(1)} \\ b_{31}^{(1)} & b_{32}^{(1)} \end{pmatrix} \quad b^{(2)} = \begin{pmatrix} b_{11}^{(2)} & b_{12}^{(2)} \\ b_{21}^{(2)} & b_{22}^{(2)} \\ b_{31}^{(2)} & b_{32}^{(2)} \end{pmatrix}$$

($t=1$)

$$V_1(1) = \pi_1 \cdot b_{Y_1^{(1)},1} \cdot b_{Y_1^{(2)},1} \quad \therefore V_1 = \begin{pmatrix} V_1(1) \\ V_1(2) \end{pmatrix} = \begin{pmatrix} \pi_1 \\ \pi_2 \end{pmatrix} \circ \begin{pmatrix} b_{Y_1^{(1)},1} \\ b_{Y_1^{(1)},2} \end{pmatrix} \circ \begin{pmatrix} b_{Y_1^{(2)},1} \\ b_{Y_1^{(2)},2} \end{pmatrix}$$

$$V_1(2) = \pi_2 \cdot b_{Y_1^{(1)},2} \cdot b_{Y_1^{(2)},2} \quad \therefore V_1 = \pi \circ \prod_{i=1}^m b_{\text{row}(Y_i^{(i)})}^T$$

($t=2 \sim T$)

$$V_2(1) = \max(V_1(1) \cdot a_{11}, V_1(2) \cdot a_{21}) \cdot b_{Y_2^{(1)},1} \cdot b_{Y_2^{(2)},1} \quad J_2(1) = 1$$

$$V_2(2) = \max(V_1(1) \cdot a_{12}, V_1(2) \cdot a_{22}) \cdot b_{Y_2^{(1)},2} \cdot b_{Y_2^{(2)},2} \quad J_2(2) = 2$$

$$\therefore V_2 = \begin{pmatrix} V_2(1) \\ V_2(2) \end{pmatrix} = \text{rowmax} \left(\begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \cdot \begin{pmatrix} V_1(1) & 0 \\ 0 & V_1(2) \end{pmatrix} \right) \circ \begin{pmatrix} b_{Y_2^{(1)},1} \\ b_{Y_2^{(1)},2} \end{pmatrix} \circ \begin{pmatrix} b_{Y_2^{(2)},1} \\ b_{Y_2^{(2)},2} \end{pmatrix}$$

$$\therefore J_2 = \begin{pmatrix} J_2(1) \\ J_2(2) \end{pmatrix} = \text{rowargmax} \left(\begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \cdot \begin{pmatrix} V_1(1) & 0 \\ 0 & V_1(2) \end{pmatrix} \right) = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$$

$$\therefore V_2 = \text{rowmax}(A^T \cdot \text{Diag}(V_1)) \circ (b_{\text{row}(Y_2^{(1)})}^T) \circ (b_{\text{row}(Y_2^{(2)})}^T) \quad \therefore J_2 = \text{rowargmax}(A^T \cdot \text{Diag}(V_1))$$

$$\therefore V_t = \text{rowmax}(A^T \cdot \text{Diag}(V_{t-1})) \circ \prod_{i=1}^m b_{\text{row}(Y_i^{(i)})}^T \quad \therefore J_t = \text{rowargmax}(A^T \cdot \text{Diag}(V_{t-1}))$$

($t=T$) $\rightarrow$ Backtracking

$$\hat{s}_{T=3} = \underset{i}{\text{argmax}} V_T(i) = 2 \text{ (EX)} \quad \hat{s}_2 = J_3(2) = 2 \quad \hat{s}_1 = J_2(2) = 2$$

$$\therefore \text{Backtracking at } T : \hat{s}_T = \underset{i}{\text{argmax}} V_T(i)$$

$$\therefore \text{Backtracking at } (T-1) \sim 1 : \hat{s}_t = J_{t+1}(\hat{s}_{t+1})$$

★ Multivariate Baum-Welch Algorithm

알고리즘 정리

E-Step : Multivariate Forward-Backward Algorithm으로 α_t, β_t 생성

0. $\gamma_t \in R^{k \times 1}$, $\xi_t \in R^{k \times k}$ 생성

$$1. \gamma_t = (\alpha_t \circ \beta_t) \cdot \frac{1}{1^T \cdot (\alpha_t \circ \beta_t)}$$

$$2. \Gamma = \begin{bmatrix} \gamma_1^T \\ \vdots \\ \gamma_T^T \end{bmatrix} \in R^{T \times k}$$

$$3. \xi_t = \text{normalize}(\text{Diag}(\alpha_t) \cdot A \cdot \text{Diag}(\beta_{t+1} \circ \bigcirc_{i=1}^m (B_{\text{row}(Y_{t+1}^{(i)})}^{(i)})^T))$$

M-Step : Update $\Pi^{new}, A^{new}, B^{(m),new}$

$$1. \Pi^{new} = \gamma_1 \in R^{k \times 1}$$

$$2. A^{new} = \left(\sum_{t=1}^{T-1} \xi_t \right) \cdot \text{Diag} \left(\sum_{t=1}^{T-1} \gamma_t \right)^{-1} \in R^{k \times k}$$

$$3. B^{(m),new} = \left(\text{Diag} \left(\sum_{t=1}^{T-1} \gamma_t \right)^{-1} \cdot \Gamma^T \cdot E^{(m)} \right)^T \in R^{N_m \times k}$$

$$| \log(1_k^T \cdot \alpha_T^{new}) - \log(1_k^T \cdot \alpha_T^{old}) | < \epsilon : \text{stop}$$

Multivariate Baum-Welch Algorithm

EX. state = $\{s_1, s_2\}$, $m (=2)$ 개의 관측 시퀀스 존재 $Y_{1:3} = \begin{pmatrix} Y_1^{(1)} \\ Y_2^{(1)} \\ Y_3^{(1)} \end{pmatrix} \begin{pmatrix} Y_1^{(2)} \\ Y_2^{(2)} \\ Y_3^{(2)} \end{pmatrix} \begin{pmatrix} Y_1^{(3)} \\ Y_2^{(3)} \\ Y_3^{(3)} \end{pmatrix}$

$$A = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \quad \Pi = \begin{pmatrix} \pi_1 \\ \pi_2 \end{pmatrix} \quad B^{(m)} = \begin{pmatrix} b_{11}^{(m)} & b_{12}^{(m)} \\ b_{21}^{(m)} & b_{22}^{(m)} \\ b_{31}^{(m)} & b_{32}^{(m)} \end{pmatrix} \quad B^{(2)} = \begin{pmatrix} b_{11}^{(2)} & b_{12}^{(2)} \\ b_{21}^{(2)} & b_{22}^{(2)} \\ b_{31}^{(2)} & b_{32}^{(2)} \end{pmatrix}$$

1 E-step: Multivariate forward-backward algorithm 오 $\alpha_t, \beta_t \in \mathbb{R}^{k \times 1}$ 계산

2 E-step: ξ_t, ζ_t 계산

$$\text{EX. } \gamma_1(2) = P(s_1 = s_2 | \text{HMM}(\lambda), Y_{\text{train}}) = \frac{\alpha_1(2) \beta_1(2)}{\sum_{j=1}^k \alpha_1(j) \beta_1(j)}$$

$$\therefore \gamma_1 = \begin{pmatrix} \gamma_1(1) \\ \gamma_1(2) \end{pmatrix} = \begin{pmatrix} \alpha_1(1) \beta_1(1) \\ \alpha_1(2) \beta_1(2) \end{pmatrix} \cdot \frac{1}{\alpha_1(1) \beta_1(1) + \alpha_1(2) \beta_1(2)} \quad \therefore \gamma_1 = \begin{pmatrix} \alpha_1(1) \\ \alpha_1(2) \end{pmatrix} \circ \begin{pmatrix} \beta_1(1) \\ \beta_1(2) \end{pmatrix} \cdot \frac{1}{\mathbf{1}^T \cdot (\alpha_1 \circ \beta_1)} = (\alpha_1 \circ \beta_1) \cdot \frac{\mathbf{1}}{\mathbf{1}^T (\alpha_1 \circ \beta_1)}$$

$$\therefore \gamma_2 = (\alpha_2 \circ \beta_2) \cdot \frac{\mathbf{1}}{\mathbf{1}^T (\alpha_2 \circ \beta_2)} \rightarrow \text{Transpose and vertical stacking } \mathcal{Y}^T = \begin{pmatrix} \gamma_1^T \\ \vdots \\ \gamma_T^T \end{pmatrix} \in \mathbb{R}^{T \times k}$$

$$\text{EX. } \xi_1(1,2) = P(s_1 = s_1, s_2 = s_2 | \text{HMM}(\lambda), Y_{\text{train}}) = \frac{\alpha_1(1) \cdot a_{12} \cdot b_{22}^{(1)} \cdot b_{12}^{(2)} \cdot \beta_2(2)}{\sum_{i=1}^k \sum_{j=1}^k \alpha_1(i) \cdot a_{ij} \cdot b_{j2}^{(1)} \cdot b_{i2}^{(2)} \cdot \beta_2(j)}$$

$$\therefore \xi_1 = \begin{pmatrix} \xi_1(1,1) & \xi_1(1,2) \\ \xi_1(2,1) & \xi_1(2,2) \end{pmatrix} : \text{unnormalize } \xi_1 = \begin{pmatrix} \alpha_1(1) & 0 \\ 0 & \alpha_1(2) \end{pmatrix} \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \cdot \text{Diag} \begin{pmatrix} b_{22}^{(1)}, 1 \\ b_{22}^{(1)}, 2 \end{pmatrix} \circ \begin{pmatrix} b_{22}^{(2)}, 1 \\ b_{22}^{(2)}, 2 \end{pmatrix} \circ \begin{pmatrix} \beta_2(1) \\ \beta_2(2) \end{pmatrix}$$

$$= \begin{pmatrix} \alpha_1(1) \cdot a_{11} & \alpha_1(1) \cdot a_{12} \\ \alpha_1(2) \cdot a_{21} & \alpha_1(2) \cdot a_{22} \end{pmatrix} \begin{pmatrix} b_{22}^{(1)} \cdot b_{22}^{(2)}, 1 & 0 \\ 0 & b_{22}^{(1)} \cdot b_{22}^{(2)}, 2 \end{pmatrix} = \begin{pmatrix} \sim & \sim \\ \sim & \sim \end{pmatrix} \begin{pmatrix} \alpha_1(1) \cdot a_{12} \cdot b_{22}^{(1)}, 2 \cdot b_{12}^{(2)}, 2 \cdot \beta_2(2) = \xi_1(1,2) \\ \sim & \sim \end{pmatrix}$$

$$\therefore \xi_1 = \text{Diag}(\alpha_1) \cdot A \cdot \text{Diag} \left(\beta_2 \circ \prod_{i=2}^m (B_{\text{row}(Y_2(i))})^T \right) : \text{unnormalize } \xi_1 \quad \therefore \xi_2 = \text{normalize} \left(\text{Diag}(\alpha_2) \cdot A \cdot \text{Diag} \left(\beta_{2+1} \circ \prod_{i=1}^m (B_{\text{row}(Y_2(i))})^T \right) \right)$$

3 M-step: Update $\Pi^{\text{new}}, A^{\text{new}}, B^{(1)\text{new}}, \dots, B^{(m)\text{new}}$

< Π^{new} update >

< A^{new} update >

$$\Pi^{\text{new}} = \gamma_1 \in \mathbb{R}^{k \times 1} \quad A^{\text{new}} = \left(\sum_{t=1}^{T-1} \xi_t \right) \cdot \text{Diag} \left(\sum_{t=1}^{T-1} \gamma_t \right)^{-1} \in \mathbb{R}^{k \times k \times k \times k}$$

< $B^{(m)\text{new}}$ update >

$$B^{(m)\text{new}} = \left(\text{Diag} \left(\sum_{t=1}^T \gamma_t \right)^{-1} \cdot \mathcal{Y}^T \cdot \mathcal{F}^{(m)} \right)^T \in \mathbb{R}^{N_m \times k}$$

$$\mathcal{Y}^T = \begin{pmatrix} \gamma_1 & \gamma_2 & \dots & \gamma_T \end{pmatrix} \in \mathbb{R}^{k \times T}$$

$$\mathcal{F}^{(m)} = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix} \in \mathbb{R}^{T \times N_m} : m\text{번 관측치의 이전 관측 개수}$$

$$\text{EX. } \mathcal{Y}^T \cdot \mathcal{F}^{(2)} = \begin{pmatrix} \gamma_1(1) & \gamma_2(1) & \gamma_3(1) \\ \gamma_1(2) & \gamma_2(2) & \gamma_3(2) \end{pmatrix} \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix} = \begin{pmatrix} \gamma_2(1) & \gamma_3(1) & \gamma_1(1) \\ \gamma_2(2) & \gamma_3(2) & \gamma_1(2) \end{pmatrix}$$

2번 관측치 시퀀스가 $(y_2^{(2)}, y_3^{(2)}, y_1^{(2)})$ 이므로

✦ Multivariate Filtering, Smoothing, Prediction

Filtering

$$f_t = P(s_t \mid y_{1:t}) = \alpha_t \cdot \frac{1}{1^T \cdot \alpha_t}$$

Smoothing

$$sm_t = P(s_t \mid y_{1:T}) = (\alpha_t \circ \beta_t) \cdot \frac{1}{1^T \cdot (\alpha_t \circ \beta_t)}$$

Prediction

$$pr_{t+h} = (A^h)^T \cdot f_t$$

🔍 Multivariate Gaussian HMM : 여러 관측값이 다변량 정규분포를 따르는 연속 확률변수인 경우

- **목표** : 다변량 정규분포를 따르는 m ($m \geq 2$)개 Observation Sequence가 HMM Input으로 들어올 때 Multivariate discrete HMM과 어떤 차이가 있는지 파악하기
- **특징**
 - 상태 전이 확률 행렬 (A), 초기 상태 확률 행렬 (Π) 은 변함이 없고,
 - 방출 확률 행렬 (B)은 다변량 정규분포 하 Observation Sequence를 생성하게 된다.
 - **행렬곱** 과 **element wise product** 로 간단하게 표현할 수 있음.

📌 Multivariate Gaussian HMM Params

Parameters

$$y_t \mid S_t = j \sim N(\mu_j, \Sigma_j) \in R^{m \times 1}$$

$$\text{State Transition Probability Matrix : } A \in R^{k \times k}$$

$$\text{Emission Probability Matrix : } b_t = \begin{bmatrix} b_t(1) = P(y_t \mid S_t = 1) \\ \vdots \\ b_t(k) = P(y_t \mid S_t = k) \end{bmatrix} \in R^{k \times 1}$$

$$\text{Initial Probability Matrix : } \Pi \in R^{k \times 1}$$

📌 Multivariate Gaussian HMM : Forward Algorithm

$$(t = 1) : \alpha_1 = \Pi \circ b_1 \in R^{k \times 1}$$

$$(t = 2 \sim T) : \alpha_t = (A^T \cdot \alpha_{t-1}) \circ b_t \in R^{k \times 1}$$

$$P(y_{test} \mid HMM(\Lambda^*)) = 1^T \cdot \alpha_T$$

📌 Multivariate Gaussian HMM : Backward Algorithm

$$(t = T) : \beta_T = 1 \in R^{k \times 1}$$

$$(t = T - 1 \sim 1) : \beta_t = A \cdot (b_{t+1} \circ \beta_{t+1}) \in R^{k \times 1}$$

$$P(y_{test} \mid HMM(\Lambda^*)) = \Pi^T \cdot (\beta_1 \circ b_1)$$

📌 Multivariate Gaussian HMM : Viterbi Algorithm

$$(t = 1) \quad v_1 = \Pi \circ b_1$$

$$(t = 2 \sim T) \quad v_t = \text{rowmax}(A^T \cdot \text{Diag}(v_{t-1})) \circ b_t \quad \tau_t = \text{rowargmax}(A^T \cdot \text{Diag}(v_{t-1})) \circ b_t$$

$$(\text{Backtracking at } T) \quad \hat{q}_T = \arg \max_i v_T(i)$$

(Backtracking at $(T - 1) \sim 1$) $\hat{q}_t = \tau_{t+1}(\hat{q}_{t+1})$

📌 Multivariate Gaussian HMM : Baum-Welch Algorithm

E-Step : Multivariate Forward-Backward Algorithm으로 α_t, β_t 생성

E-step : $\gamma_t \in R^{k \times 1}, \xi_t \in R^{k \times k}$ 생성

$$1. \gamma_t = (\alpha_t \circ \beta_t) \cdot \frac{1}{1^T \cdot (\alpha_t \circ \beta_t)}$$

$$2. \Gamma = \begin{bmatrix} \gamma_1^T \\ \vdots \\ \gamma_T^T \end{bmatrix} \in R^{T \times k}$$

$$3. \xi_t = \frac{\text{Diag}(\alpha_t) \cdot A \cdot \text{Diag}(\beta_{t+1} \circ b_{t+1})}{1_k^T \cdot \text{Diag}(\alpha_t) \cdot A \cdot \text{Diag}(\beta_{t+1} \circ b_{t+1}) \cdot 1_k}$$

M-Step : Update $\Pi^{new}, A^{new}, \{\mu_j, \Sigma_j\}_{j=1}^k$

$$1. \Pi^{new} = \gamma_1 \in R^{k \times 1}$$

$$2. A^{new} = \left(\sum_{t=1}^{T-1} \xi_t \right) \cdot \text{Diag} \left(\sum_{t=1}^{T-1} \gamma_t \right)^{-1} \in R^{k \times k}$$

$$3. \mu_j^{new} = \frac{\sum_{t=1}^T \gamma_t(j) y_t}{\sum_{t=1}^T \gamma_t(j)} \quad \mu^{new} = \text{Diag}(1_T^T \cdot \Gamma)^{-1} \cdot \Gamma^T \cdot Y \in R^{k \times m} \quad \text{where } Y =$$

$$\mu^{new} \text{의 } j \text{ 행} = \mu_j^T$$

$$4. \Sigma_j^{new} = \frac{\sum_{t=1}^T \gamma_t(j) \cdot (y_t - \mu_j) \cdot (y_t - \mu_j)^T}{\sum_{t=1}^T \gamma_t(j)} = (Y - 1_T \cdot \mu_{row(j)}^{new})^T \cdot \text{Diag}(\Gamma_{col(j)}) \cdot (Y$$

$$| \log(1_k^T \cdot \alpha_T^{new}) - \log(1_k^T \cdot \alpha_T^{old}) | < \epsilon : \text{stop}$$

Multivariate Gaussian HMM에서 사용되는 알고리즘의 핵심 요약

Forward Algorithm : 과거의 추적

- 시작점부터 현재시점까지의 모든 상태 전이 경로 확률과 각 시점의 관측 확률을 누적합산하여 현재 상태에 도달할 전방향 확률을 결정한다.
- $(1 \sim t)$ 현재 시점까지의 모든 상태 전이 경로 확률과 관측값 확률이 반영되어 있다.

Backward Algorithm : 미래의 예견

- 현재 시점에서 특정 상태 i 에 있다고 가정할 때, 그 이후부터 마지막 시점까지의 모든 상태전이 경로와 관측값이 나타날 후방향 확률을 구하며, 이는 Forward Algorithm과 결합하여 특정 시점의 상태를 추론하는 근거가 된다.
- $(t + 1 \sim T)$ 현재 시점의 전이 경로와 관측값 확률을 포함되지 않으며, 미래 전이 경로와 관측값의 확률만 반영되어 있다.

Viterbi Algorithm : 최적의 경로만 선택

- Forward Algorithm과 달리 전체 관측 시퀀스를 생성했을 가능성이 가장 큰 단일 최적 상태 경로를 찾기 위해, 각 단계에서 최대 확률을 기록하고 마지막에 역추적을 수행한다.

Baum-Welch Algorithm : 기대치 기반의 자기 개선

- E-step
 - γ_t : Forward(α)와 Backward(β) 정보를 결합하여, 전체 관측 데이터가 주어졌을 때 t 시점에 상태 i 에 머물러 있을 사후 확률을 의미한다.
 - ξ_t : t 시점에 상태 i 이고, $t + 1$ 시점에 상태 j 로 전이했을 두 시점 간의 결합 사후 확률을 의미한다.
- M-step
 - A update : $T - 1$ 시점까지 발생한 상태 i 에서 j 로의 전이 횟수 기대치를 상태 i 에 머문 횟수 기대치로 나누어 업데이트한다.
 - μ, Σ update : 각 상태에 속할 확률(γ_t)을 가중치로 삼아, 전체 학습 데이터(Y)의 가중 평균과 가중 공분산을 구하여 파라미터를 최적화한다.

- 수렴 조건 : 업데이트된 파라미터로 계산한 **전체 로그 우도(Log-Likelihood)**가 이전 단계보다 개선되었는지 확인하며, 개선 폭이 임계값(ϵ)보다 작아질 때까지 반복한다.

Multivariate Gaussian HMM : 직접 구현 시 고려해야 할 사항

Underflow

문제점

- Forward/Backward Algorithm에서는 시점 t 가 지남에 따라 상태 전이 확률과 그 상태에서의 관측값의 확률을 지속적으로 곱함.
- 이때 확률은 1보다 작은 값이므로 확률들의 누적곱은 컴퓨터가 부동 소수점으로 표현할 수 있는 최소 범위보다 작아져 0으로 인식할 수 있는 문제점이 존재.
- 이는 Learning Problem을 위한 Baum-Welch Algorithm 학습이 불가능해지는 결과를 초래할 수 있음.

해결책 1: **Scaling**

- **정의** : 특정 t 시점의 각 상태별 전방확률 또는 후방확률의 총합을 나누어서 t 시점의 상태 별 전방확률 또는 후방확률이 0 ~ 1 사이의 적절한 크기를 유지하게 하여 0으로 소멸되는 것을 방지
- **적용 알고리즘** : Forward / Backward / Baum-Welch Algorithm
 - 이때 우도를 구하는 과정에서는 로그 우도를 활용해야 한다.
- Scaling Forward Algorithm

$$(t = 1) : \alpha_1^{unscaled} = \Pi \circ b_1 \in R^{k \times 1}$$

$$(t = 1) : \alpha_1^{scaled} = \frac{\alpha_1^{unscaled}}{1_k^T \cdot \alpha_1^{unscaled}} \in R^{k \times 1}$$

$$(t = 2 \sim T) : \alpha_t^{unscaled} = (A^T \cdot \alpha_{t-1}^{scaled}) \circ b_t \in R^{k \times 1}$$

$$(t = 2 \sim T) : \alpha_t^{scaled} = \frac{\alpha_t^{unscaled}}{1_k^T \cdot \alpha_t^{unscaled}} \in R^{k \times 1}$$

$$\ln P(y_{test} \mid HMM(\Lambda^*)) = \sum_{t=1}^T \ln(1_k^T \cdot \alpha_t^{unscaled})$$

- Scaling Backward Algorithm

$$(t = T) : \beta_T^{scaled} = 1 \in R^{k \times 1}$$

$$(t = T - 1 \sim 1) : \beta_t^{unscaled} = A \cdot (b_{t+1} \circ \beta_{t+1}^{scaled}) \in R^{k \times 1}$$

$$(t = T - 1 \sim 1) : \beta_t^{scaled} = \frac{\beta_t^{unscaled}}{1_k^T \cdot \beta_t^{unscaled}} \in R^{k \times 1}$$

$$\ln P(y_{test} \mid HMM(\Lambda^*)) = \ln(\Pi^T \cdot (b_1 \circ \beta_1^{scaled})) + \sum_{t=1}^{T-1} \ln(1_k^T \cdot \beta_t^{unscaled})$$

- Baum-Welch Algorithm에서는 Scaled Forward / Backward Probability를 사용한다.

해결책 2 : [Log-transformation](#)

- **정의** : 확률의 곱 대신 log summation 을 활용하여 시점 t가 지속됨에 따라 0으로 수렴하는 것을 방지
- **적용 알고리즘** : Viterbi Algorithm
- log Viterbi Algorithm

$$(t = 1) \quad \ln(v_1) = \ln(\Pi) + \ln(b_1) \in R^{k \times 1}$$

$$(t = 2 \sim T) \quad \ln(v_t) = \text{rowmax}(\ln(A^T) + 1_k \cdot \ln(v_{t-1}^T)) + \ln(b_t) \quad \tau_t = \text{rowargm}$$

$$(\text{Backtracking at } T) \quad \hat{q}_T = \arg \max_i \ln(v_T(i))$$

$$(\text{Backtracking at } (T - 1) \sim 1) \quad \hat{q}_t = \tau_{t+1}(\hat{q}_{t+1})$$

✚ 공분산 행렬의 Singularity

문제점

- 특정 시점의 Emission Probability vector를 구하고자 할 때, 다변량 정규분포 PDF $b_t(j) = \frac{1}{(2\pi)^{m/2} |\Sigma_j|^{1/2}} \exp\left(-\frac{1}{2}(y_t - \mu_j)^T \Sigma_j^{-1} (y_t - \mu_j)\right)$ 계산과정에서 $\text{Det}(\Sigma_j)$ 가 0이 되어 역행렬을 구하지 못하는 문제

해결책

- Baum-Welch Algorithm의 Σ_j 업데이트 과정에서 다음을 수행한다.

$$\Sigma_j^{new} = \Sigma_j^{new} + \epsilon \cdot I \quad \epsilon \approx 1e-6$$

✚ 수치적 안정성을 위한 Log PDF 계산

문제점

$$b_t(j) = \frac{1}{(2\pi)^{m/2} |\Sigma_j|^{1/2}} \exp\left(-\frac{1}{2}(y_t - \mu_j)^T \Sigma_j^{-1} (y_t - \mu_j)\right)$$

- 다변량 정규분포의 PDF를 이용하여 관측 확률을 구하는 과정에서 관측값이 평균과 멀어지는 경우 $\exp(\cdot)$ 계산 결과는 0에 수렴할 수 있다.
- 한번 0이 되면 로그를 취해도 $-\infty$ 가 되어 전방 / 후방 확률을 계산 과정이 망가질 수 있다.

해결책

- b_t 계산을 로그 다변량 정규분포 PDF로부터 구한다.

$$\ln(b_t(j)) = \underbrace{-\frac{m}{2} \ln(2\pi)}_{\text{상수항}} \underbrace{-\frac{1}{2} \ln |\Sigma_j|}_{\text{행렬식 항}} \underbrace{-\frac{1}{2} (y_t - \mu_j)^T \Sigma_j^{-1} (y_t - \mu_j)}_{\text{마할라노비스 거리 항}}$$

$$b_t(j) = \exp(\ln(b_t(j)))$$

🚩 Zero Transition

문제점

- Baum-Welch Algorithm의 M-step에서 전이 행렬 A 를 업데이트 할 때, 특정 상태 i 에서 j 로 가는 전이가 학습 데이터에서 단 한 번도 관찰되지 않으면 A_{ij} 값이 정확히 0이 되는 현상
- 이는 학습 데이터의 양이 충분하지 않아 모델이 가능한 모든 전이 케이스를 경험하지 못했을 때 발생하며 (데이터 희소성, Sparse Data)
- 한 번 0이 된 확률은 다음 Iteration에서도 계속 0으로 남게 되어 해당 경로는 영원히 차단되는 죽은 경로가 됨. (모델의 경직성 유발 원인)

해결책 : Laplace Smoothing

- 학습 데이터에 특정 상태 간의 전이가 한 번도 나타나지 않았더라도, 그 확률을 완전히 0으로 만들지 않기 위해서 Laplace Smoothing 을 활용.
- Baum-Welch Algorithm의 A 업데이트 과정에서 다음을 수행한다.

$$A_{ij}^{new} = \frac{\sum_{t=1}^{T-1} \xi_t(i, j) + \delta}{\sum_{t=1}^{T-1} \gamma_t(i) + k \cdot \delta} \quad \rightarrow \quad A^{new} = \left(\sum_{t=1}^{T-1} \xi_t + \delta \cdot \mathbf{1}_{k \times k} \right) \cdot \text{Diag} \left(\sum_{t=1}^{T-1} \gamma_t + \delta \cdot \mathbf{1}_k \right)^{-1}$$

🔴 파라미터 초기화

문제점

- 다변량 정규분포 HMM의 Baum-Welch 알고리즘에서 가장 중요한 것은 실제 데이터 환경에 맞게 State 별 평균과 공분산을 잘 추정하는지의 여부이다.
- 이때 Baum-Welch 알고리즘은 지역 최소점 (Local Minimum)에 빠질 수 있기 때문에 적절한 초기값을 부여하지 않으면 올바른 결론을 낼 수 있다.

해결책 : K-means

- K-means 알고리즘으로 구한 클러스터 별 평균과 공분산을 초기값으로 사용
- Baum-Welch Algorithm의 Initailize 단계에서 다음의 K-means 결과값을 초기값으로 지정한다.

$$\mu_j = \frac{1}{N_j} \sum_{y_t \in C_j} y_t$$

$$\Sigma_j = \frac{1}{N_j} \sum_{y_t \in C_j} (y_t - \mu_j)(y_t - \mu_j)^T \quad \rightarrow \quad \Sigma_j = \Sigma_j + \epsilon I \quad (\epsilon \approx 1e-6)$$

🔴 Final Algorithms of Multivariate Gaussian HMM

- $\ln(b_t(j)) = -\frac{m}{2} \ln(2\pi) - \frac{1}{2} \ln |\Sigma_j| - \frac{1}{2} (y_t - \mu_j)^T \Sigma_j^{-1} (y_t - \mu_j)$: Viterbi Algorithm 에서 사용
- $b_t(j) = \exp(\ln(b_t(j)))$: Forward / Backward Algorithm 에서 사용

Forward Algorithm

$$(t = 1) : \alpha_1^{unscaled} = \Pi \circ b_1 \in R^{k \times 1}$$

$$(t = 1) : \alpha_1^{scaled} = \frac{\alpha_1^{unscaled}}{\mathbf{1}_k^T \cdot \alpha_1^{unscaled}} \in R^{k \times 1}$$

$$(t = 2 \sim T) : \alpha_t^{unscaled} = (A^T \cdot \alpha_{t-1}^{scaled}) \circ b_t \in R^{k \times 1}$$

$$(t = 2 \sim T) : \alpha_t^{scaled} = \frac{\alpha_t^{unscaled}}{1_k^T \cdot \alpha_t^{unscaled}} \in R^{k \times 1}$$

$$\ln P(y_{test} \mid HMM(\Lambda^*)) = \sum_{t=1}^T \ln(1_k^T \cdot \alpha_t^{unscaled})$$

Backward Algorithm

$$(t = T) : \beta_T^{scaled} = 1 \in R^{k \times 1}$$

$$(t = T - 1 \sim 1) : \beta_t^{unscaled} = A \cdot (b_{t+1} \circ \beta_{t+1}^{scaled}) \in R^{k \times 1}$$

$$(t = T - 1 \sim 1) : \beta_t^{scaled} = \frac{\beta_t^{unscaled}}{1_k^T \cdot \beta_t^{unscaled}} \in R^{k \times 1}$$

$$\ln P(y_{test} \mid HMM(\Lambda^*)) = \ln(\Pi^T \cdot (b_1 \circ \beta_1^{scaled})) + \sum_{t=1}^{T-1} \ln(1_k^T \cdot \beta_t^{unscaled})$$

Viterbi Algorithm

$$(t = 1) \quad \ln(v_1) = \ln(\Pi) + \ln(b_1) \in R^{k \times 1}$$

$$(t = 2 \sim T) \quad \ln(v_t) = \text{rowmax}(\ln(A^T) + \mathbf{1}_k \cdot \ln(v_{t-1}^T)) + \ln(b_t) \quad \tau_t = \text{rowargm}$$

$$(\text{Backtracking at } T) \quad \hat{q}_T = \arg \max_i \ln(v_T(i))$$

$$(\text{Backtracking at } (T - 1) \sim 1) \quad \hat{q}_t = \tau_{t+1}(\hat{q}_{t+1})$$

Baum-Welch Algorithm

$$\text{E-Step : K-means Algorithm으로 } \{\mu_j = \frac{1}{N_j} \sum_{y_t \in C_j} y_t, \Sigma_j = \frac{1}{N_j} \sum_{y_t \in C_j} (y_t - \mu_j)(y_t - \mu_j)$$

$$\text{E-Step : Scaling Forward-Backward Algorithm으로 } \alpha_t, \beta_t \text{ 생성} \quad (\alpha_t = \alpha_t^{scaled}, \beta_t$$

$$\text{E-step : } \gamma_t \in R^{k \times 1}, \xi_t \in R^{k \times k} \text{ 생성}$$

$$1. \gamma_t = (\alpha_t \circ \beta_t) \cdot \frac{1}{\mathbf{1}^T \cdot (\alpha_t \circ \beta_t)}$$

$$2. \Gamma = \begin{bmatrix} \gamma_1^T \\ \vdots \\ \gamma_T^T \end{bmatrix} \in R^{T \times k}$$

$$3. \xi_t = \frac{\text{Diag}(\alpha_t) \cdot A \cdot \text{Diag}(\beta_{t+1} \circ b_{t+1})}{\mathbf{1}_k^T \cdot \text{Diag}(\alpha_t) \cdot A \cdot \text{Diag}(\beta_{t+1} \circ b_{t+1}) \cdot \mathbf{1}_k}$$

M-Step : Update $\Pi^{new}, A^{new}, \{\mu_j, \Sigma_j\}_{j=1}^k$

$$1. \Pi^{new} = \gamma_1 \in R^{k \times 1}$$

$$2. A_{ij}^{new} = \frac{\sum_{t=1}^{T-1} \xi_t(i, j) + \delta}{\sum_{t=1}^{T-1} \gamma_t(i) + k \cdot \delta} \quad \rightarrow \quad A^{new} = \left(\sum_{t=1}^{T-1} \xi_t + \delta \cdot \mathbf{1}_{k \times k} \right) \cdot \text{Diag} \left(\sum_{t=1}^{T-1} \gamma_t \right)$$

$$3. \mu_j^{new} = \frac{\sum_{t=1}^T \gamma_t(j) y_t}{\sum_{t=1}^T \gamma_t(j)} \quad \mu^{new} = \text{Diag}(\mathbf{1}_T^T \cdot \Gamma)^{-1} \cdot \Gamma^T \cdot Y \in R^{k \times m} \quad \text{where } Y =$$

$$\mu^{new} \text{의 } j \text{ 행} = \mu_j^T$$

$$4. \Sigma_j^{new} = \frac{\sum_{t=1}^T \gamma_t(j) \cdot (y_t - \mu_j) \cdot (y_t - \mu_j)^T}{\sum_{t=1}^T \gamma_t(j)} = (Y - \mathbf{1}_T \cdot \mu_{row(j)}^{new})^T \cdot \text{Diag}(\Gamma_{col(j)}) \cdot (Y$$

$$\Sigma_j^{new} = \Sigma_j^{new} + \epsilon \cdot I \quad \epsilon \approx 1e-6$$

$$|\mathcal{L}_{new} - \mathcal{L}_{old}| < \epsilon : \text{stop} \quad \mathcal{L} = \sum_{t=1}^T \ln(\mathbf{1}_k^T \cdot \alpha_t^{unscaled})$$

HMM 요약 정리

알고리즘 핵심 간단 설명

Forward Algorithm

- Observation Sequence와 HMM 파라미터가 주어졌을 때, Observation Sequence의 모든 시점에 대하여 상태 전이 경로와 그 때의 관측 확률을 **모두** 고려한 **하나의 Likelihood 값** 을 도출하는 알고리즘
- Observation Sequence와 HMM 파라미터 구조만 있으면 수행할 수 있는 알고리즘이므로, **Train Data 와 Test Data에 대해서 모두 수행** 할 수 있음.
- 이때 상태 전이 경로와 관측 확률의 요약을 **$t = 1$ 시점부터 고려** 함.
- **가장 과거 시점부터 특정 시점까지의** 모든 상태 전이 경로와 그때의 관측 확률을 고려한 하나의 요약 확률을 **전방확률(Forward Probability)** 이라고 정의함.

Backward Algorithm

- Observation Sequence와 HMM 파라미터가 주어졌을 때, Observation Sequence의 모든 시점에 대하여 상태 전이 경로와 그 때의 관측 확률을 **모두** 고려한 **하나의 Likelihood 값** 을 도출하는 알고리즘
- Observation Sequence와 HMM 파라미터 구조만 있으면 수행할 수 있는 알고리즘이므로, **Train Data 와 Test Data에 대해서 모두 수행** 할 수 있음.
- 이때 상태 전이 경로와 관측 확률의 요약을 **$t = T$ 시점부터 고려** 함.
- **가장 미래 시점부터 특정 시점까지의** 모든 상태 전이 경로와 그때의 관측 확률을 고려한 하나의 요약 확률을 **후방확률(Backward Probability)** 이라고 정의함.

Baum-Welch Algorithm

- Train Data Observation Sequence 와 HMM architecture 가 주어졌을 때, Train Data에 대해서 최적의 HMM Parameter Set을 찾는 학습 알고리즘
- Test Data 에 대해서 적용하면 선견 편향 (Look Ahead Bias) 이(가) 발생할 수 있어, **Train Data에 대해서만 수행** 할 수 있음.

- EM 알고리즘의 일종이므로 E-Step과 M-Step 이 존재.
- E-Step
 - HMM architecture 가 주어져 있으므로, Forward / Backward Algorithm을 이용하여 Forward Probability 와 Backward Probability 를 계산
 - 이때 전방확률과 후방확률을 모두 계산하는 이유는, 학습 과정에서 전체 Observation Sequence에 대해 가장 설명력이 좋은 Parameter Set을 찾아야 하기 때문.
 - 이는 앞/뒤 어느 측면에서 봤을 때도 가장 그럴듯한 결과 를 내야한다는 의미.
 - 앞서 구한 전방 확률과 후방 확률을 이용해서 $\gamma_t \in R^{k \times 1}$ 와 $\xi_t \in R^{k \times k}$ 를 구함.
 - 이때 γ_t 의 개별 값은 t 시점에서 상태가 i 일 확률, ξ_t 의 개별 값은 t 시점에서 상태가 i 이고 $t + 1$ 에서 상태가 j 일 확률을 의미.
 - γ_t 는 상태 별 t 시점의 전방확률과 후방확률의 상대적 비율로 계산.
 - ξ_t 는 상태 i 의 t 시점 전방확률과 상태 j 의 $t + 1$ 시점 후방 확률 그리고 그 사이를 연결해주는 $t + 1$ 시점에서의 상태 전이 확률과 $t + 1$ 시점에서의 관측 확률 사이의 상대적 비율로 계산.
 - E-step 에서 구한 γ_t 와 ξ_t 는 M-step에서 파라미터의 최적값을 구하는 재료.
- M-Step : HMM의 파라미터는 모든 학습 기간을 고려했을 때 가장 설명력이 높아야 하므로, 기본적으로 모든 시점의 γ_t 와 ξ_t 를 고려.
 - 상태 전이 확률 행렬 (A) Update
 - 모든 시점에서 상태 i 에 있을 확률 $\gamma_t(i)$ 를 더한 값과, 모든 시점에서 상태 i 에서 j 로 전이될 확률 $\xi_t(i, j)$ 를 더한 값의 상대적 비율로 $A_{i,j}$ 계산.
 - 초기 상태 확률 행렬 (Π) Update
 - $t = 1$ 시점에서 상태 i 에 있을 확률이므로, γ_1 을 그대로 사용함.
 - 상태 별 정규분포 Parameter (평균 μ , 공분산 Σ) Update
 - 모든 기간의 상태 별 $\gamma_t(j)$ 에 대한 평균 가중합, 공분산 가중합으로 계산.
- EM 알고리즘으로 지속적으로 Parameter를 Update 한 후, Forward Algorithm 으로 구한 Likelihood 값이 더 이상 개선되지 않을 때까지 반복.

Viterbi Algorithm

- Observation Sequence와 HMM 파라미터가 주어졌을 때, 최적의 (가장 설명력이 높은) 상태 전이 경로가 무엇인지를 찾아내는 알고리즘
- 최적의 경로를 제시하기 위한 알고리즘이기 때문에, Train Data 보다는 학습이 완료된 후, Test Data에 대해 수행하는 것이 일반적임.
- Viterbi Algorithm은 실시간으로 최적의 경로를 찾는 방식이 아님.
- Observation Sequence를 살펴보는 과정에서 "내가 지금(t) 상태(i) 에 있다고 생각했을 때(t 시점에서 Viterbi 확률이 큰 것) , 어떤 상태에서 온 것이 더 확률이 높을까? (τ_t)" 를 모든 상태 전이 과정과 관측 확률을 고려해서 기록함.
- 그 후 모든 Observation Sequence를 살펴본 후 최종적으로 확률이 가장 높은 상태를 확정짓고, 다시 역순으로 "내가 지금 상태 i 에 있다고 생각하면 상태 j 에서 온게 최선이구나" 같이 미리 기록해둔 상태 전이 시나리오를 역으로 파악하는(Backtracking) 방식으로 최적의 상태 전이 경로를 제시함.
- 따라서 Viterbi가 제시하는 최적의 상태 전이 경로는 미래의 상태까지 고려한 것이므로, 노이즈를 잘 필터링 할 수 있음.
- 예를 들어 상승 추세에서 약간의 노이즈가 발생하여 하락장 같은 모습이 나왔다고 할지라도, 미래에서 상승 추세가 유지된다면 이전 상태는 하락추세 상태가 아닌 상승 추세에서 왔을거라 판단할 수 있음. (상태 지속성에 의해 -> A의 대각 성분 확률이 큰 경우)

Viterbi algorithm : Backtracking 오직 최적의 경로를 찾아내는 이유

Ex. $k=2$ 상태 1: 상음장 U, 상태 2: 하음장 D, $t=1 \sim 3$

$$A = \begin{pmatrix} 0.9 & 0.1 \\ 0.1 & 0.9 \end{pmatrix} : \begin{array}{l} \text{상음장일 때 상음장이 유지될 가능성이 높고} \\ \text{하음장일 때 하음장이 유지될 가능성이 높다.} \end{array} \quad \text{state persistence가 높음} \quad \pi = \begin{pmatrix} 1.0 \\ 0 \end{pmatrix}$$

시간 별 상태 별 전방 확률

t	U	D	
1	0.9	0.1	$\rightarrow$ 전 방향 상음장 선호
2	0.05	0.95	하음장 //
3	0.95	0.05	상음장 //

주준히 올라가다가 어느 하향 곳 조정이
왔는데 그 다음날 다시 상음 추세 회복
 $\therefore$ 상음 추세 속 약간의 노이즈가 발생한 경우

< Greedy Search >

($t=1$)

$$V_1(1) = 1 \times 0.9 = 0.9 : U \text{ 선택}$$

$$V_1(2) = 0 \times 0.1 = 0$$

($t=2$)

$$\begin{array}{l} \text{상태 U} \left\{ \begin{array}{l} 0.9 \times 0.05 = 0.045 \\ 0.1 \times 0.95 = 0.095 : D \text{ 선택} \end{array} \right. \end{array}$$

($t=3$)

$$\begin{array}{l} \text{상태 D} \left\{ \begin{array}{l} 0.1 \times 0.95 = 0.095 : U \text{ 선택} \\ 0.9 \times 0.05 = 0.045 \end{array} \right. \end{array}$$

$$\therefore U \rightarrow D \rightarrow U$$

Greedy Search 방식은 시노이즈를 만들 경우

노이즈에 민감하게 반응할 수 있음

< Viterbi Backtracking >

($t=1$)

$$V_1(1) = 1 \times 0.9 = 0.9$$

$$V_1(2) = 0 \times 0.1 = 0$$

($t=2$)

$$V_2(1) = \max \begin{pmatrix} 0.9 \times 0.9 \\ 0 \times 0.1 \end{pmatrix} \cdot 0.05 = 0.0405$$

$$V_2(2) = \max \begin{pmatrix} 0.9 \times 0.1 \\ 0 \times 0.9 \end{pmatrix} \cdot 0.95 = 0.0855$$

($t=3$)

$$V_3(1) = \max \begin{pmatrix} 0.0405 \times 0.9 \\ 0.0855 \times 0.1 \end{pmatrix} \cdot 0.95 = 0.0346$$

$$V_3(2) = \max \begin{pmatrix} 0.0405 \times 0.1 \\ 0.0855 \times 0.9 \end{pmatrix} \cdot 0.05 = 0.0038$$

$$\hat{x}_3 = 1 \quad \hat{x}_2 = 1 \quad \hat{x}_1 = 1$$

$\therefore U \rightarrow U \rightarrow U$ (상음 추세의 노이즈를 잘 필터링함)

Filtering

- 현재 t 시점의 모든 상태 별 전방확률의 합에 대한 특정 상태 j 의 전방확률의 비율을 제시하여, 현재 시점의 각 상태 별 확률을 계산하는 방법
- Filtering이 제시하는 확률로 argmax를 이용해 filtering path를 제시하는 경우 시점 마다의 노이즈에 민감하게 반응할 수 있음.
- 이는 Filtering이 미래를 보지 못하고 과거부터 현재까지의 결과로만 판단하기 때문.

Smoothing

- 특정 t 시점의 모든 상태 별 전방확률 $\times$ 후방확률의 합에 대한 특정 상태 j 의 전방확률 $\times$ 후방확률의 비율을 제시하여, 특정 시점의 각 상태 별 확률을 계산하는 방법
 - Baum-Welch Algorithm 에서 사용한 γ_t 계산 방식과 동일

- Smoothing이 제시하는 확률로 argmax 를 이용해 smoothing path를 제시하는 경우 시점 마다의 노이즈를 Filtering 보다 잘 필터링 할 수 있음.
- 이는 전방확률과 후방확률을 활용해 과거와 미래를 모두 살펴본 후, 현재를 결정하기 때문.

Prediction

- 현재 Filtering 확률을 기준으로 $\mathcal{A}$ 를 활용해 h 시점 이후의 상태 별 확률을 계산하는 방법
- Prediction 또한 Filtering과 마찬가지로 미래를 보고 판단하지 않기 때문에 노이즈에 민감할 수 있음.